Data Engineering Project

Trip Ingestion
Data Engineering Project

Local architecture for scalable ingestion

Challenge Objective

Architect a scalable, containerized local pipeline for automated trip data ingestion and real-time reporting.

March 2026

Technical Interview

Challenge Requirements

Acceptance Criteria

Pipeline Scope

Architect a robust data engineering solution focused on performance and observability, operating entirely in a local environment without managed cloud dependencies.

Target Volume

100M+

Trip records processed

Automated Ingestion

Asynchronous processing of large CSV files via REST API.

Logical Grouping

Aggregation by origin, destination, and specific time windows.

Regional Metrics

Weekly average trip calculations filtered by region or bounding box.

Real-time Status

Immediate progress feedback via Server-Sent Events (no polling).

SQL Database

Structured and indexed relational persistence (PostgreSQL).

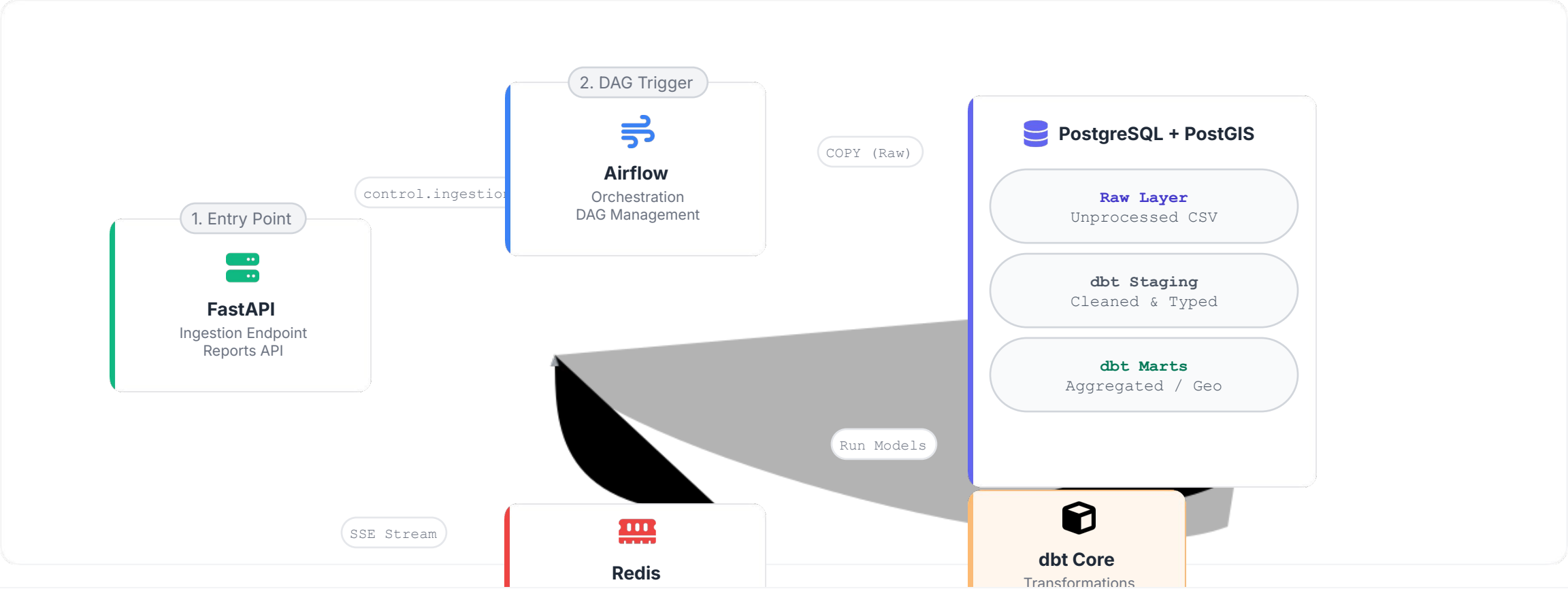
Containerization

Reproducible local execution orchestrated via Docker Compose.

System Overview

Proposed Architecture

End-to-end flow: Ingestion, Transformation, and Real-time Feedback



Local Stack Components

- Docker Compose
- Apache Airflow
- FastAPI
- PostgreSQL / PostGIS
- dbt Core
- Redis

Tech Stack

Components & Responsibilities

Infrastructure details and service roles

Orchestrator

Apache Airflow

Orchestrates complex DAGs
Manages retries & backfills
Monitors ingestion SLAs

API Gateway

FastAPI

Exposes REST endpoints
Validates contracts (Pydantic)
Handles Auth & Rate Limiting

Warehouse

PostgreSQL + PostGIS

Schemas: raw, staging, marts
Table partitioning strategies
Geospatial indexing (GiST)

Transformation

dbt Core

Modular SQL modeling
Medallion architecture implementation
Data quality testing

Cache/PubSub

Redis

Pub/sub for progress events
Backend for Server-Sent Events
Short-term caching

Infrastructure

Docker Compose

Local stack provisioning
Network isolation
Environment variable injection

Data Pipeline

Ingestion & Transformation Flow

Event-driven orchestration from upload to analytics

Ingestion Phase (EL)

1 API Request

POST /ingestions registers request, validates metadata schema, and returns a unique ingestion ID.

2 Airflow Trigger

DAG triggers, temporarily downloads CSV file, and validates basic file integrity.

3 Bulk Load (Raw)

Executes Postgres COPY command for high-throughput loading into *raw* tables.

4 dbt Staging

Bronze/Silver

Data cleaning, strong typing (timestamps, floats), and initial deduplication logic.

✓ Idempotent ✓ Scalable

Transform & Serving Phase (T)

5 dbt Marts

Gold Layer

Business aggregations: weekly averages, time windows, and regional grouping logic.

6 Event Publishing

Airflow/dbt hooks publish progress and completion events to Redis Pub/Sub channels.

7 Real-time Feedback

No Polling

Client receives live status updates via Server-Sent Events (SSE) consumed from Redis.

8 Query Mart Tables

GET /reports API queries optimized Mart tables, delivering low-latency analytical data.

Scalability & Conclusion

Production Ready

Cloud-Portable Design

The local architecture mirrors enterprise patterns. Containerization and decoupled orchestration allow for immediate migration to managed services (AWS RDS, MWAA, ECS) without refactoring.

Cloud Service Architecture Components -GCP Example

The same architecture we can achieve using GCP services as Cloud Composer, Kafka Pub/sub, BigQuery and DBT Cloud as outside provider.

COPY-based Ingestion

Uses COPY for high-performance stream writing, avoiding individual insert overhead.

Partitioning

Tables partitioned by trip_datetime to optimize temporal scans and data management.

Spatial Indexing

PostGIS indices (GiST/BRIN) accelerate complex bounding box queries and geometry.

Incremental Modeling

Incremental dbt models process only new data, reducing computation time and cost.

Pre-aggregated Marts

Materialized Gold layer optimized for fast reads, isolating analytical load from Raw.

Segregated API

Reports API queries only the Marts, ensuring low latency and high availability.